Name: Satorre, Lance Timothy B.
GitHub Link: https://github.com/Lancesatorre/IntProgActivities/tree/main/Lab4-Activity_SATORRE/role-based-app-backend

1. Project Overview

This project is a Role-Based CRUD (Create, Read, Update, Delete) Web Application that demonstrates authentication, authorization, and data management across multiple entities such as users, employees, departments, and requests.

The system differentiates between admin and regular users**,** enforcing access control and UI restrictions based on roles.

2. Purpose of the System

The primary goal of this application is to:

- Demonstrate role-based access control (RBAC)
- Implement CRUD operations across multiple modules
- Showcase authentication using JWT
- Simulate a full-stack system using lightweight technologies

3. Technology Stack

Backend

- **Node.js** - runtime environment
- **Express.js** - web framework for routing and middleware

Frontend

- **HTML5** - structure
- **CSS3** - styling
- **Vanilla JavaScript** - client-side logic and DOM manipulation

Database / Storage

- **JSON file (data.json)** - acts as a simple persistent data store

4. System Architecture

The application follows a client-server architecture:

- **Frontend (Client)** communicates with the backend via REST APIs
- **Backend (Server)** processes requests and interacts with the JSON database
- **Authentication Layer** secures endpoints using JWT

5. Authentication Flow

1. **User Registration**
    - Users create an account via `/api/register`
2. **User Login**
    - Credentials are validated via `/api/login`
    - A **JWT token** is generated
3. **Session Handling**
    - Token is stored in `sessionStorage`
    - Used for authenticated API requests
4. **Role-Based Access**
    - **Admin**: full access (users, employees, departments, requests)
    - **User**: limited access (own profile, requests)
5. **Password Update**
    - Available via `/api/profile/password`

7. Data Model

All data is stored in a single JSON file (`data.json`) with the following structure:

Entities:

- **Users**
    - id
    - username

    - password
    - role

- **Employees**
    - id
    - name
    - department
    - position

- **Departments**
    - name
    - description

- **Requests**
    - id
    - userId
    - type
    - status

8. API Endpoints

Authentication

- **POST** /api/register – Register new user
- **POST** /api/login – Login user
- **PUT** /api/profile/password – Update password

Users

- **GET** /api/users/:id – Get user
- **PUT** /api/users/:id – Update user
- **DELETE** /api/users/:id – Delete user

Employees

- **GET** /api/employees – Get all employees
- **POST** /api/employees – Create employee
- **PUT** /api/employees/:id – Update employee
- **DELETE** /api/employees/:id – Delete employee

Departments

- **GET** /api/departments – Get all departments
- **POST** /api/departments – Create department
- **PUT** /api/departments/:name – Update department
- **DELETE** /api/departments/:name – Delete department

Requests

- **GET** /api/requests – Get all requests
- **POST** /api/requests – Create request
- **PUT** /api/requests/:id – Update request
- **DELETE** /api/requests/:id – Delete request

9. Frontend Pages

The application consists of the following pages:

- **Home Page** – Landing interface
- **Login/Register Page** – Authentication UI
- **Profile Page** – User account management
- **Employees Page** – Employee CRUD interface
- **Departments Page** – Department management
- **Accounts Page** – User management (admin only)
- **Requests Page** – Request handling system

10. Key Code Components

Backend

- **Server Setup & Routing**

```node_modules/express/index.js
const path = require('path');
const express = require('express');
const cors = require('cors');
const { PORT } = require('./config');

const authRoutes = require('./routes/authRoutes');
const userRoutes = require('./routes/userRoutes');
const employeeRoutes = require('./routes/employeeRoutes');
const departmentRoutes = require('./routes/departmentRoutes');
const requestRoutes = require('./routes/requestRoutes');

const app = express();

app.use(cors());
app.use(express.json());
app.use(express.static(path.join(__dirname, 'public')));

app.use('/api', authRoutes);
app.use('/api', userRoutes);
app.use('/api', employeeRoutes);
app.use('/api', departmentRoutes);
app.use('/api', requestRoutes);

app.get('/', (req, res) => {
    res.sendFile(path.join(__dirname, 'public', 'FullStack.html'));
});

app.listen(PORT, () => {
    console.log(`✅ Backend running on http://localhost:${PORT}`);
});
```

Shows route mounting and static file serving

- **Data Store & Persistence**

```node_modules/express/index.js
const fs = require('fs');
const bcrypt = require('bcryptjs');
const { DATA_FILE } = require('../config');

const defaultData = () => ({
    users: [
        {
            id: 1,
            first: 'Admin',
            last: 'User',
            email: 'admin@example.com',
            password: bcrypt.hashSync('Password123!', 10),
            role: 'admin',
            verified: true
        }
    ],
    employees: [
        { id: '001', first: 'Jordan', last: 'Avery', position: 'Lead Engineer', dept: 'Engineering', hireDate: '2024-02-12' },
        { id: '002', first: 'Abby', last: 'Torres', position: 'Success Manager', dept: 'Customer Success', hireDate: '2023-11-03' }
    ],
    requests: []
});

function loadData() {
    if (fs.existsSync(DATA_FILE)) {
        try {
            return JSON.parse(fs.readFileSync(DATA_FILE, 'utf8'));
        } catch (err) {
            console.warn('Could not parse data file, resetting to defaults', err);
        }
    }
    const data = defaultData();
    fs.writeFileSync(DATA_FILE, JSON.stringify(data, null, 2));
    return data;
}

let dataStore = loadData();

function saveData() {
    fs.writeFileSync(DATA_FILE, JSON.stringify(dataStore, null, 2));
}

function getStore() {
    return dataStore;
}

module.exports = {
    getStore,
    saveData,
    defaultData
};
```

Handles reading/writing JSON data

- **Authentication Logic**

```
const express = require('express');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const { SECRET_KEY } = require('../config');
const { getStore, saveData } = require('../data/store');

const router = express.Router();
const dataStore = getStore();

router.post('/register', async (req, res) => {
    const { first, last, email, password, role, verified } = req.body;

    if (!email || !password || !first || !last) {
        return res.status(400).json({ error: 'All fields are required' });
    }

    const existing = dataStore.users.find(u => u.email === email);
    if (existing) {
        return res.status(409).json({ error: 'Email already exists' });
    }

    const hashedPassword = await bcrypt.hash(password, 10);
    const newUser = {
        id: dataStore.users.length + 1,
        first,
        last,
        email,
        password: hashedPassword,
        role: role || 'user',
        verified: typeof verified === 'boolean' ? verified : true
```

```
router.post('/login', async (req, res) => {
    const { email, password } = req.body;

    const user = dataStore.users.find(u => u.email === email);
    if (!user || !(await bcrypt.compare(password, user.password))) {
        return res.status(401).json({ error: 'Invalid credentials' });
    }

    const token = jwt.sign(
        { id: user.id, email: user.email, role: user.role, first: user.first, last: user.last },
        SECRET_KEY,
        { expiresIn: '1h' }
    );

    res.json({ token, user: { email: user.email, role: user.role, first: user.first, last: user.last } });
});
```

```
router.put('/profile/password', async (req, res) => {
    const { email, newPassword } = req.body;
    if (!email || !newPassword) {
        return res.status(400).json({ error: 'Email and new password are required' });
    }

    const user = dataStore.users.find(u => u.email === email);
    if (!user) {
        return res.status(404).json({ error: 'User not found' });
    }

    user.password = await bcrypt.hash(newPassword, 10);
    saveData();
    res.json({ message: 'Password updated successfully' });
});

module.exports = router;
```

Includes register, login, password update

- **Employee Routes**

```
const express = require('express');
const { getStore, saveData } = require('../data/store');

const router = express.Router();
const dataStore = getStore();

router.get('/employees', (req, res) => {
    res.json({ employees: dataStore.employees });
});

router.post('/employees', (req, res) => {
    const { id, first, last, email, position, dept, hireDate } = req.body;
    if (!id || !first || !last || !email || !position || !dept || !hireDate) {
        return res.status(400).json({ error: 'All employee fields are required' });
    }

    const newEmployee = { id, first, last, position, dept, hireDate, email };
    dataStore.employees.push(newEmployee);
    saveData();
    res.status(201).json({ employee: newEmployee });
});

router.get('/employees/:id', (req, res) => {
    const emp = dataStore.employees.find(e => e.id === req.params.id);
    if (!emp) {
        return res.status(404).json({ error: 'Employee not found' });
    }
    res.json({ employee: emp });
});

router.put('/employees/:id', (req, res) => {
    const { first, last, position, dept, email, hireDate } = req.body;
    const emp = dataStore.employees.find(e => e.id === req.params.id);
    if (!emp) {
        return res.status(404).json({ error: 'Employee not found' });
    }
    if (first) emp.first = first;
    if (last) emp.last = last;
    if (position) emp.position = position;
    if (dept) emp.dept = dept;
    if (email) emp.email = email;
    if (hireDate) emp.hireDate = hireDate;
    saveData();
    res.json({ employee: emp });
});

router.delete('/employees/:id', (req, res) => {
    const idx = dataStore.employees.findIndex(e => e.id === req.params.id);
    if (idx === -1) {
        return res.status(404).json({ error: 'Employee not found' });
    }
    const removed = dataStore.employees.splice(idx, 1)[0];
    saveData();
    res.json({ employee: removed });
});
```

CRUD operations including delete

- **Department Routes**

Lab4-Activity_SATORRE > role-based-app-backend > routes > departmentRoutes.js > ...

```
const express = require('express');
const { getStore, saveData } = require('../data/store');

const router = express.Router();
const dataStore = getStore();

router.get('/departments', (req, res) => {
    res.json({ departments: dataStore.departments });
});

router.post('/departments', (req, res) => {
    const { name, description } = req.body;
    if (!name || !description) {
        return res.status(400).json({ error: 'Department name and description are required' });
    }

    const newDepartment = { name, description };
    dataStore.departments.push(newDepartment);
    saveData();
    res.status(201).json({ department: newDepartment });
});

router.put('/departments/:name', (req, res) => {
    const { name, description } = req.body;
    const dept = dataStore.departments.find(d => d.name === req.params.name);
    if (!dept) {
        return res.status(404).json({ error: 'Department not found' });
    }
    if (name) dept.name = name;
    if (description) dept.description = description;
    saveData();
    res.json({ department: dept });
});

router.delete('/departments/:name', (req, res) => {
    const idx = dataStore.departments.findIndex(d => d.name === req.params.name);
    if (idx === -1) {
        return res.status(404).json({ error: 'Department not found' });
    }
    const removed = dataStore.departments.splice(idx, 1)[0];
    saveData();
    res.json({ department: removed });
});

module.exports = router;
```

- **Request Routes**

```
const express = require('express');
const { getStore, saveData } = require('../data/store');

const router = express.Router();
const dataStore = getStore();

router.get('/requests', (req, res) => {
    res.json({ requests: dataStore.requests });
});

router.get('/requests/:id', (req, res) => {
    const reqObj = dataStore.requests.find(r => r.id === parseInt(req.params.id));
    if (!reqObj) {
        return res.status(404).json({ error: 'Request not found' });
    }
    res.json({ request: reqObj });
});

router.put('/requests/:id', (req, res) => {
    const { status, type, items, createdBy } = req.body;
    const reqObj = dataStore.requests.find(r => r.id === parseInt(req.params.id));
    if (!reqObj) {
        return res.status(404).json({ error: 'Request not found' });
    }
    if (status) reqObj.status = status;
    if (type) reqObj.type = type;
    if (Array.isArray(items) && items.length) {
        reqObj.items = items;
        reqObj.itemsCount = items.length;
    }
    if (createdBy) reqObj.createdBy = createdBy;
    saveData();
    res.json({ request: reqObj });
});

router.delete('/requests/:id', (req, res) => {
    const id = parseInt(req.params.id);
    const idx = dataStore.requests.findIndex(r => r.id === id);
    if (idx === -1) {
        return res.status(404).json({ error: 'Request not found' });
    }
    const removed = dataStore.requests.splice(idx, 1)[0];
    saveData();
    res.json({ request: removed });
});

router.post('/requests', (req, res) => {
    const { type, items, status, createdBy } = req.body;
    if (!type || !Array.isArray(items) || !items.length) {
        return res.status(400).json({ error: 'Request type and at least one item are req
    }

    const newRequest = {
        id: dataStore.requests.length + 1,
        type,
        items,
        itemsCount: items.length,
        status: status || 'Pending',
        createdBy: createdBy || 'unknown'
    };
    dataStore.requests.push(newRequest);
    saveData();
    res.status(201).json({ request: newRequest });
});

module.exports = router;
```

Frontend

1. **API & Auth Helpers**

Lab4-Activity_SATORRE > role-based-app-backend > public > JS api.js > ...

```javascript
(function (window) {
    const API_BASE = 'http://localhost:3000';
    const auth = window.Auth;

    async function request(path, { method = 'GET', headers = {}, body } = {}) {
        const response = await fetch(API_BASE + path, {
            method,
            headers: {
                'Content-Type': 'application/json',
                ...(auth?.getAuthHeader?.() || {}),
                ...headers
            },
            body
        });
        const data = await response.json().catch(() => ({}));
        if (!response.ok) {
            throw new Error(data.error || data.message || 'Request failed');
        }
        return data;
    }

    const postJson = (path, payload, method = 'POST') => request(path, { method, body: JSON.stringify(payload) });

    const fetchDepartments = () => request('/api/departments');
    const fetchEmployees = () => request('/api/employees');
    const fetchAccounts = () => request('/api/users');
    const fetchRequests = () => request('/api/requests');

    window.ApiClient = {
        request,
        postJson,
        fetchDepartments,
        fetchEmployees,
        fetchAccounts,
        fetchRequests
    };
})(window);
```

Lab4-Activity_SATORRE > role-based-app-backend > public > JS auth.js >

```javascript
(function (window) {
    const TOKEN_KEY = 'authToken';
    const USER_KEY = 'authUser';

    function parseJwt(token) {
        try {
            return JSON.parse(atob(token.split('.')[1]));
        } catch (err) {
            return null;
        }
    }

    function getToken() {
        return sessionStorage.getItem(TOKEN_KEY);
    }

    function getStoredUser() {
        const raw = sessionStorage.getItem(USER_KEY);
        return raw ? JSON.parse(raw) : null;
    }

    function setStoredUser(user) {
        if (user) {
            sessionStorage.setItem(USER_KEY, JSON.stringify(user));
        } else {
            sessionStorage.removeItem(USER_KEY);
        }
    }

    function setSession(token, user) {
        if (token) {
            sessionStorage.setItem(TOKEN_KEY, token);
        } else {
            sessionStorage.removeItem(TOKEN_KEY);
        }
        setStoredUser(user);
    }

    function clearSession() {
        setSession(null, null);
    }

    function getAuthHeader() {
        const token = getToken();
        return token ? { Authorization: `Bearer ${token}` } : {};
    }

    window.Auth = {
        parseJwt,
        getToken,
        getStoredUser,
        setStoredUser,
        setSession,
        clearSession,
        getAuthHeader
    };
})(window);
```

2. **App Initialization & Routing**
    - I can't screenshot it but just look at **FullStack.js:1–120**
3. **Employees Table UI**
    - I can't screenshot it but just look at **FullStack.js:40–90**
4. **Employee CRUD Handlers**
    - I can't screenshot it but just look at **FullStack.js:310–380**
5. **Requests UI Actions**
    - I can't screenshot it but just look at **FullStack.js:500–620**

11. Key Features Summary

- Role-based authentication (Admin/User)
- Full CRUD functionality across modules
- JWT-based session handling
- Simple JSON-based persistence
- Dynamic frontend with modular UI

12. Limitations

- No real database (uses JSON file only)
- No password hashing (if not implemented)
- Limited scalability
- Basic UI/UX

13. Recommendations for Improvement

- Integrate **MongoDB / MySQL**
- Add **password hashing (bcrypt)**
- Implement **refresh tokens**
- Improve UI using frameworks (React, Vue)
- Add **search, filtering, pagination**
- Deploy using cloud platforms